

RELATÓRIO TEORIA DOS GRAFOS - PARTE 1

Implementação de biblioteca de grafos não-direcionados

Estudante 01	Laissa Rosa dos Santos (9390)
Estudante 02	Ana Flávia Tavares Alves (8772)
URL do repositório	laissarosa/Graph-Theory-SIN-430-: Repository for the SIN 430 Course - Graph Theory, with implementations of undirected graphs

1. Decisões de projeto e implementação

A biblioteca foi implementada em Python, orientada a objetos, baseado em uma classe abstrata (Graph) e duas classes concretas que a especializam: AdjacencyListGraph e AdjacencyMatrixGraph, além da classe GraphStatistics que calcula e formata as estatísticas para um objeto Graph. Os vértices são numerados de 1 a n , seguindo o formato do arquivo de entrada da disciplina, e não de 0 a $n-1$ como é comum em estruturas internas do Python. Essa escolha evita a necessidade de conversão de índices em toda a interface pública da biblioteca, tornando o uso mais direto para quem consome a biblioteca.

Na representação por lista de adjacência, cada vértice mantém um conjunto (set) com seus vizinhos, em vez de uma lista. Essa escolha garante que a operação de adicionar uma aresta seja $O(1)$ em média e que arestas duplicadas no arquivo de entrada sejam automaticamente ignoradas, sem necessidade de verificação explícita a cada inserção. O custo de memória dessa representação é $O(V+E)$, pois cada aresta é armazenada apenas nas duas listas dos vértices que ela conecta.

Na representação por matriz de adjacência, cada linha da matriz é armazenada como um bytearray de tamanho $V+1$, em vez de uma lista de inteiros do Python. Essa escolha foi motivada por uma questão de memória: um bytearray ocupa exatamente 1 byte por posição, enquanto uma lista de inteiros do Python ocupa dezenas de bytes por elemento (cada inteiro é um objeto Python completo, com overhead próprio). Ainda assim, o custo total de memória da matriz é $O(V^2)$, pois toda a matriz $V \times V$ é alocada independentemente de quantas arestas o grafo realmente possui, por isso, essa representação se mostra inviável para grafos grandes e esparsos, como discutido na seção de resultados.

A leitura do arquivo de entrada foi implementada como um método (from_file), definido uma única vez na classe abstrata Graph e herdado pelas duas subclasses. Ao ser chamado a partir de AdjacencyListGraph.from_file(...) ou AdjacencyMatrixGraph.from_file(...), o método constrói automaticamente uma instância da classe correta, a escolha da representação é feita simplesmente escolhendo qual classe se chama, sem necessidade de parâmetros adicionais ou de estruturas condicionais para selecionar a representação. Durante a leitura, arestas do tipo laços são identificadas e ignoradas, já que o trabalho define grafos simples e não-direcionados; da mesma forma, arestas duplicadas não geram erro nem são contabilizadas mais de uma vez.

2. Desempenho em termos de quantidade de memória e tempo de execução utilizada pelas duas representações

Os resultados de consumo de memória confirmam o comportamento assintótico esperado das duas representações. A matriz de adjacência armazena uma estrutura de tamanho $V \times V$, com custo de memória $O(V^2)$, enquanto a lista de adjacência armazena apenas as arestas de fato existentes, com custo $O(V+E)$. Nos Grafos 01 e 02, ambas as representações ainda couberam na memória disponível, a partir do Grafo 03, a construção da matriz de adjacência resultou em erro de memória (MemoryError), pois o número de vértices desses grafos é grande o suficiente para que o espaço V^2 necessário ultrapasse a memória RAM (8 GB) disponível na máquina utilizada nos testes. Para esses casos, o espaço necessário foi calculado através da fórmula V^2 , onde V é o número de vértices do grafo, como a matriz possui uma linha e uma coluna para cada vértice, o número total de células é $V \times V$, e como cada célula ocupa 1 byte na implementação utilizada, o resultado em bytes é diretamente V^2 , convertido em GB ou TB. A lista de adjacência, por outro lado, continuou operando normalmente em todos os grafos, conforme apresentado na Tabela 1.

Grafo / Representação	Matriz de Adjacência	Lista de Adjacência
Grafo 01	99.355 MB	29.129 MB
Grafo 02	2470.598 MB	280.988 MB
Grafo 03	130.97 GB (aproximado)	197.109 MB
Grafo 04	130.97 GB (aproximado)	1145.527 MB
Grafo 05	21.34 TB (aproximado)	2701.477 MB
Grafo 06	21.34 TB (aproximado)	3456.016 MB

Tabela 1: Representação aproximada da memória usada por cada grafo.

2.1 Tempo médio de buscas - em largura (BFS) e em profundidade (DFS)

Para o algoritmo de busca em largura, os resultados do tempo de execução estão descritos na Tabela 2.

Grafo / Representação	Matriz de Adjacência	Lista de Adjacência
Grafo 01	6.640180 segundos	0.042332 segundos
Grafo 02	33.369321 segundos	0.256424 segundos
Grafo 03	<i>Memory Error</i>	0.311988 segundos
Grafo 04	<i>Memory Error</i>	3.924337 segundos
Grafo 05	<i>Memory Error</i>	6.045059 segundos
Grafo 06	<i>Memory Error</i>	764.401033 segundos

Tabela 2: Representação do tempo médio por busca em largura.

Em relação ao algoritmo de busca em profundidade, os resultados do tempo de execução se encontram descritos na Tabela 3.

Grafo / Representação	Matriz de Adjacência	Lista de Adjacência
Grafo 01	6.127943 segundos	0.065633 segundos
Grafo 02	36.769202 segundos	0.502780 segundos
Grafo 03	<i>Memory Error</i>	0.506940 segundos
Grafo 04	<i>Memory Error</i>	7.589611 segundos
Grafo 05	<i>Memory Error</i>	18.289088 segundos
Grafo 06	<i>Memory Error</i>	520.511998 segundos

Tabela 3: Representação do tempo médio por busca em profundidade.

3. Pai dos vértices nas árvores geradoras

Nos casos em que a coluna correspondente apresenta o símbolo "-", isso indica que o vértice em questão não foi alcançado a partir do vértice inicial da busca, ou seja, os dois vértices pertencem a componentes conexas distintas do grafo. Esse resultado é consistente com o número de componentes conexas identificados na Tabela 6. No Grafo 03, que possui duas componentes conexas, observa-se que, ao iniciar a busca no vértice 1, apenas o vértice 30 é alcançado, permanecendo os vértices 10 e 20 sem pai definido; já ao iniciar a busca nos vértices 2 ou 3, o padrão se inverte, com os vértices 10 e 20 sendo alcançados e o vértice 30 permanecendo inacessível. Isso indica que o vértice 1 está na mesma componente conexa do vértice 30, enquanto os vértices 2, 3, 10 e 20 pertencem a outra componente. No Grafo 05, que possui cinco componentes conexas, o vértice 20 não é alcançado a partir de nenhum dos três vértices iniciais testados (1, 2 e 3), o que indica que ele pertence a uma componente conexa distinta das componentes que contêm esses três vértices. Os dados completos dessa análise para os algoritmos BFS e DFS estão organizados, respectivamente, nas Tabelas 4 e 5.

Pai dos vértices 10,20,30 a partir de diferentes vértices iniciais	Busca em Largura								
	Vértice 10			Vértice 20			Vértice 30		
	Vértice Inicial 1	Vértice Inicial 2	Vértice Inicial 3	Vértice Inicial 1	Vértice Inicial 2	Vértice Inicial 3	Vértice Inicial 1	Vértice Inicial 2	Vértice Inicial 3
grafo 1	2042	8935	7685	8382	5762	4656	3555	3491	5783
grafo 2	-	21365	-	-	-	46738	-	-	21931
grafo 3	-	158403	158403	-	75471	319691	141597	-	-
grafo 4	194465	-	-	28623	-	-	136244	-	-
grafo 5	1888350	-	1888350	-	-	-	2502539	-	4692602

grafo 6	-	1677854	-	-	1124114	-	-	4231681	-
---------	---	---------	---	---	---------	---	---	---------	---

Tabela 4: Representação do pai dos vértices na busca em largura.

Pai dos vértices 10,20,30 a partir de diferentes vértices iniciais	Busca em Profundidade								
	Vértice 10			Vértice 20			Vértice 30		
	Vértice Inicial 1	Vértice Inicial 2	Vértice Inicial 3	Vértice Inicial 1	Vértice Inicial 2	Vértice Inicial 3	Vértice Inicial 1	Vértice Inicial 2	Vértice Inicial 3
grafo 1	4349	3446	4349	7541	7541	7541	2685	2685	2685
grafo 2	-	6119	-	-	-	887	-	-	45946
grafo 3	-	106718	60012	-	141526	141526	141597	-	-
grafo 4	237822	-	-	316278	-	-	1531	-	-
grafo 5	1928249	-	1928249	-	-	-	4692602	-	4692602
grafo_6	-	1701759	-	-	2638973	-	-	3605754	-

Tabela 5: Representação do pai dos vértice na busca em largura.

4. Distância entre pares de vértices

A ocorrência de "não há caminho" entre determinados pares de vértices decorre do mesmo fenômeno observado na questão anterior: os vértices envolvidos pertencem a componentes conexas diferentes. Esse resultado é coerente com os dados já apresentados nas Tabelas 4 e 6. No Grafo 03, por exemplo, não há caminho entre os pares (10,30) e (20,30), o que está de acordo com a árvore de busca da Questão 4, em que o vértice 30 não foi alcançado a partir de buscas iniciadas nos vértices 2 e 3 (mesma componente de 10 e 20). No Grafo 05, a ausência de caminho entre (10,20) e (20,30) é igualmente consistente com o fato de o vértice 20 pertencer a uma componente conexa distinta da dos vértices 10 e 30. A existência de caminho e as respectivas distâncias entre os pares de vértices analisados estão detalhadas na Tabela 6.

Grafo / Pares	(10, 20)	(10, 30)	(20, 30)
Grafo 01	3	3	4
Grafo 02	Não há caminho	Não há caminho	3
Grafo 03	9	Não há caminho	Não há caminho
Grafo 04	4	3	4
Grafo 05	Não há caminho	9	Não há caminho
Grafo 06	5	5	5

Tabela 6: Representação da distância entre os pares dos vértices.

5. Componentes conexas do grafo

O Grafo 01 é o único, entre os grafos já processados, que se mostrou totalmente conexo. Os Grafos 03 e 05, por sua vez, apresentam 2 e 5 componentes conexas, respectivamente, com tamanhos bastante diferentes. No Grafo 05, a maior componente concentra 2.500.000 vértices, enquanto a menor possui apenas 156.250, demonstrando uma distribuição desigual do tamanho das componentes nesse grafo.

Os valores de diâmetro foram obtidos pelo cálculo aproximado e são relativamente pequenos mesmo em grafos com centenas de milhares ou milhões de vértices (por exemplo, diâmetro 19 no Grafo 03 e diâmetro 11 no Grafo 05). Os dados sobre as componentes conexas e os diâmetros de cada grafo estão na Tabela 7.

Grafo / Pares	Quantidade de componentes	Maior componente	Menor componente	Diâmetro
Grafo 01	1	10000	0	5
Grafo 02	10	25000	48	5
Grafo 03	2	250000	125000	19
Grafo 04	2	250000	125000	4
Grafo 05	5	2500000	156250	11
Grafo 06	5	2500000	156250	7

Tabela 7: Representação das componentes conexas e do diâmetro dos grafos.